

Backend Developer — Step 2

SQL & Database Fundamentals

Roadmap objective: Learn relational databases and SQL well enough to design and query backend data

Level: Beginner → Intermediate

Last reviewed: September 2026

How to use these notes

Read the concepts first, reproduce the examples yourself, complete the practical lab, and answer the interview questions without looking at the notes. Use the official documentation links as the final authority for changing APIs and platform behavior.

Learning outcomes

- Database vs DBMS
- Tables, rows, columns and keys
- CRUD SQL
- Constraints
- Joins
- GROUP BY/HAVING
- Transactions and indexes
- Basic schema design

Core concepts

1. A backend developer needs to understand how application data is stored, queried and constrained.
2. Primary keys identify rows; foreign keys express relationships between tables.
3. Indexes improve lookup performance but add write/storage cost; use them intentionally.
4. Transactions group related operations into an all-or-nothing unit.

Concept map

```
SQL & Database Fundamentals
|
+--> Learn the concept
|
+--> Reproduce a small example
|
+--> Apply it in a feature
|
+--> Test failure/edge cases
|
+--> Document the decision
```

Detailed study guide

1. Database vs DBMS

Study database vs dbms through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

2. Tables, rows, columns and keys

Study tables, rows, columns and keys through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

3. CRUD SQL

Study crud sql through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

4. Constraints

Study constraints through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

5. Joins

Study joins through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

6. GROUP BY/HAVING

Study group by/having through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

Worked example

SQL table + query

```
CREATE TABLE users (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL  
);  
  
SELECT id, name  
FROM users  
WHERE email LIKE '%@example.com'  
ORDER BY name;
```

Practice variation: change one input, introduce one failure, predict the result, then run it. Rewrite the example from memory afterward.

Comparison table

Concept	Meaning	Practical use
Primary key	Uniquely identifies a row	Usually indexed
Foreign key	References another table	Represents relationship
Index	Speeds selected reads	Costs storage/write work

Common mistakes

- Using SELECT * everywhere
- Skipping constraints
- Adding indexes without understanding queries
- Mixing unrelated data in one table
- Ignoring transactions for multi-step updates

Best practices

- Keep responsibilities clear and small.
- Validate inputs at system boundaries.
- Prefer readable code over clever code.
- Test important success and failure paths.
- Use official documentation for current API/platform behavior.
- Keep secrets and credentials out of source control.
- Document important trade-offs so another developer can reproduce your reasoning.

Extended practical lab

Complete these tasks in order. The goal is to turn passive knowledge into evidence that you can actually use the topic.

Lab 1

- Create a schema on paper before writing SQL.
- Write down what you expected, what happened, and why.

Lab 2

- Write INSERT, SELECT, UPDATE and DELETE statements.
- Write down what you expected, what happened, and why.

Lab 3

- Write one INNER JOIN and one LEFT JOIN.
- Write down what you expected, what happened, and why.

Lab 4

- Add a UNIQUE and NOT NULL constraint and intentionally violate each.
- Write down what you expected, what happened, and why.

Lab 5

- Create one index and explain which query it should help.
- Write down what you expected, what happened, and why.

Lab 6

- Wrap two related writes in a transaction and test rollback.
- Write down what you expected, what happened, and why.

Mini-project

Design a small Career Tracker database with users, careers, roadmap_steps and progress tables. Write CRUD, joins, aggregation and transaction examples.

Acceptance criteria

- A new developer can understand the project from its README.
- The main happy path works from start to finish.
- At least three edge/failure cases are tested.
- The project has meaningful Git commits.
- The project includes a short explanation of design choices.
- If deployment applies, production behavior is verified rather than assumed.

Interview preparation

Practice answering these aloud. A strong answer should define the concept, give a concrete example, mention one trade-off, and explain when you would use it.

Primary key vs foreign key?

- Answer structure: definition → example → trade-off/limitation → practical use.

INNER JOIN vs LEFT JOIN?

- Answer structure: definition → example → trade-off/limitation → practical use.

Why do indexes help reads?

- Answer structure: definition → example → trade-off/limitation → practical use.

What is a transaction?

- Answer structure: definition → example → trade-off/limitation → practical use.

What is normalization?

- Answer structure: definition → example → trade-off/limitation → practical use.

When can an index hurt performance?

- Answer structure: definition → example → trade-off/limitation → practical use.

Debugging checklist

- Reproduce the problem consistently.
- Reduce it to the smallest failing example.
- Inspect inputs at each boundary.
- Check the first incorrect value, not only the final error.
- Read the complete error message and stack trace.
- Change one thing at a time.
- Retest the original failure after the fix.
- Add a regression test when the bug is important.

Glossary

Term	Your own definition	Example/use
DBMS	Write this in your own words.	Give one project example.
Schema	Write this in your own words.	Give one project example.
Primary key	Write this in your own words.	Give one project example.
Foreign key	Write this in your own words.	Give one project example.
Constraint	Write this in your own words.	Give one project example.
Join	Write this in your own words.	Give one project example.
Index	Write this in your own words.	Give one project example.
Transaction	Write this in your own words.	Give one project example.

Term	Your own definition	Example/use
Normalization	Write this in your own words.	Give one project example.
Query	Write this in your own words.	Give one project example.

Self-test

- Explain Database vs DBMS without using its textbook definition.
- Show a small example of Tables, rows, columns and keys.
- What is the most important boundary in this topic?
- What is one failure mode and how would you diagnose it?
- What trade-off should a developer know before using this technique?
- How would you test the normal path and an edge case?
- How does this topic connect to the next roadmap step?
- What would you change if the system had 100× more users/data?
- What part of this topic would you explain differently to a beginner?
- What can you now build that you could not build before studying this step?

Final checklist

- I can explain and demonstrate Database vs DBMS.
- I can explain and demonstrate Tables, rows, columns and keys.
- I can explain and demonstrate CRUD SQL.
- I can explain and demonstrate Constraints.
- I can explain and demonstrate Joins.
- I can explain and demonstrate GROUP BY/HAVING.
- I can explain and demonstrate Transactions and indexes.
- I can explain and demonstrate Basic schema design.
- I completed the practical lab.
- I built or extended the mini-project.
- I tested at least three failure/edge cases.
- I can explain one trade-off.
- I can answer the interview questions without notes.
- I know where to find the authoritative documentation.

Recommended documentation

- <https://dev.mysql.com/doc/>

Recommended videos

- CodeWithHarry — SQL Tutorial for Beginners (Complete Course using MySQL)
<https://www.youtube.com/watch?v=yE6tIle64tU> (NEW)

Source note: resource references are based on the existing CareerCompass database plus verified web research. For changing APIs, versions, deployment settings and security guidance, consult the linked official documentation before production use.